

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Athisaya U	Reg.No.:	192571001
Department:	B.Tech Computer Science and Biosciences	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I / Athisaya U / 192571001 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Athisaya U

QUESTION 1 — DMA-Based Device-to-Memory Transfer

Goal: move a block from an I/O device into main memory with one CPU setup phase and an interrupt-driven completion path.

Problem Understanding

Inputs	Device data register, DMA-safe destination address, and byte count
Outputs	A filled memory buffer plus SUCCESS or FAILURE status
Constraint	The CPU configures the DMA controller but must not copy individual bytes

Design Rationale

DMA is appropriate for a large or repeated device-to-memory transfer because the controller becomes the bus master after the CPU supplies the source, destination, count, and transfer direction. This keeps the processor free for other tasks while the payload moves directly into RAM.

The interrupt handler is intentionally short. It reads the completion status, acknowledges the interrupt, records an error when required, and signals the waiting task. Processing the received data remains outside interrupt context so interrupt latency stays bounded.

Process Flow

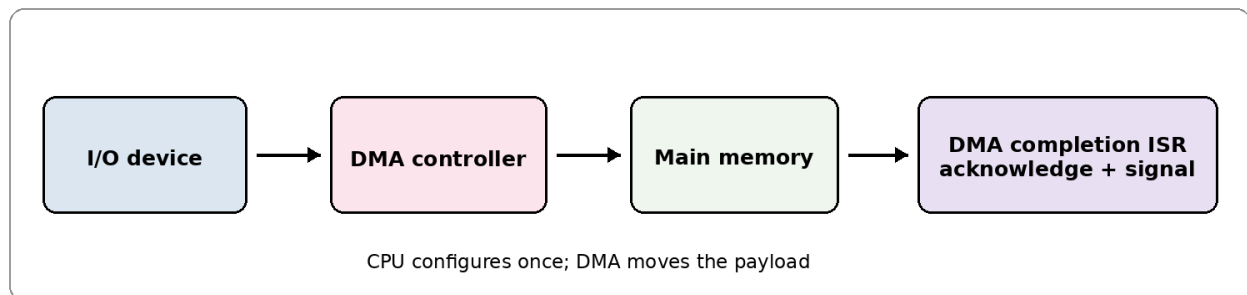


Figure 1. DMA payload path and short completion interrupt path.

Execution Sequence

1. Validate that the destination buffer is DMA-safe before programming the controller.
2. Load the DMA registers once, enable completion and error interrupts, then start the transfer.
3. Suspend the waiting task instead of polling the DMA status register in a CPU loop.
4. On completion or fault, let the ISR update state and return control to the waiting task.

Algorithm

PSEUDOCODE

```
ALGORITHM DMA_TRANSFER(device, destination, count)
  REQUIRE count > 0 AND DMA_SAFE(destination)
  done <- FALSE; error <- NONE
  DMA.SOURCE <- device.DATA_REGISTER
  DMA.DESTINATION <- destination
  CONFIGURE DMA(count, DEVICE_TO_MEMORY)
  ENABLE DMA_INTERRUPTS(COMPLETE, ERROR)
  START DMA
  WHILE done = FALSE AND error = NONE DO
    WAIT_FOR_INTERRUPT()
  END WHILE
  IF error = NONE THEN
    RETURN SUCCESS
  ELSE
    RETURN FAILURE
  END IF
END ALGORITHM

INTERRUPT_HANDLER DMA_ISR()
  status <- READ DMA.STATUS
  ACKNOWLEDGE DMA_INTERRUPT
  IF status = COMPLETE THEN
    done <- TRUE
  ELSE
    error <- status; STOP DMA
    RECORD_ERROR(status)
  END IF
  SIGNAL waiting task
  RETURN_FROM_INTERRUPT
END HANDLER
```

Validation and Efficiency

Efficiency note: the CPU writes a small set of control registers once, then sleeps until an interrupt reports completion or failure. No CPU copy loop runs while the device payload is moving.

VALIDATION / EDGE CASES

Test or condition	Expected behaviour
DMA-safe buffer and valid count	Start transfer; the payload is copied by DMA, not by the CPU.
Count is zero or buffer is invalid	Reject the request before enabling the DMA controller.
DMA status = COMPLETE	Set done flag, acknowledge interrupt, and wake the waiting task.
DMA error or bus fault	Stop DMA, record the status, and return FAILURE safely.

RESULT: DMA moves the payload directly, so the CPU handles only setup and a short completion/error interrupt.

QUESTION 2 — Priority-Based Vectored Interrupt Handling

Goal: always service the highest-priority pending interrupt and dispatch its device-specific routine directly through a vector table.

Problem Understanding

Inputs	Pending vectors, controller priority levels, and vector table entries
Outputs	Correct ISR execution with the interrupt mask restored
Constraint	Mask only lower priorities while the selected ISR is executing

Design Rationale

A vectored interrupt controller supplies the identifier of the highest-priority device that is currently requesting service. The processor therefore uses the vector table to jump directly to the appropriate service routine instead of testing every device status register.

The priority mask prevents a low-priority interrupt from interrupting a critical service routine, while a later request is still retained by the controller. Each ISR performs only the minimum device-specific work and posts longer work to a deferred task.

Process Flow

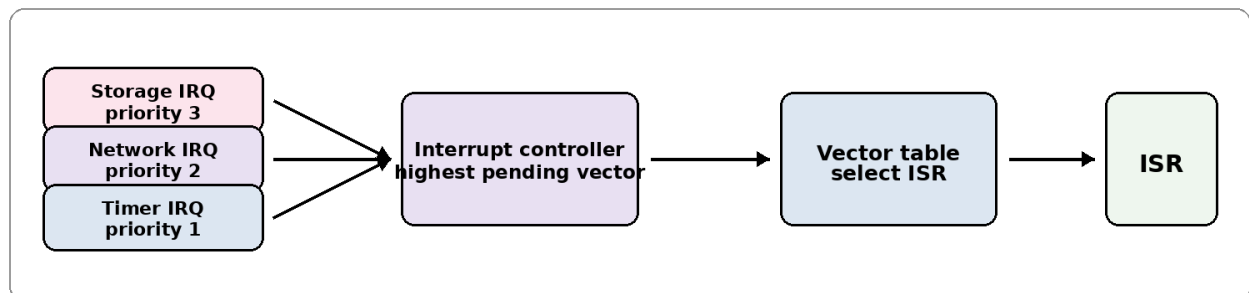


Figure 2. Direct vector selection for priority-ordered device interrupts.

Execution Sequence

1. Read the highest pending vector from the interrupt controller.
2. Validate the vector, mask lower priorities, acknowledge, and invoke its ISR.
3. Restore the interrupt mask and move longer work to a deferred task.

Algorithm

PSEUDOCODE

```
ALGORITHM DISPATCH_VECTORED_INTERRUPTS()
```

```
  LOOP FOREVER
```

```
    vector <- HIGHEST_PENDING_VECTOR()
```

```
    IF vector = NONE THEN
```

```
      WAIT_FOR_INTERRUPT()
```

```
      CONTINUE
```

```
    END IF
```

```
    entry <- VECTOR_TABLE[vector]
```

```
    IF entry = INVALID THEN
```

```
      CALL DEFAULT_ERROR_ISR()
```

```
      CONTINUE
```

```
    END IF
```

```
    MASK_LOWER_PRIORITIES(entry.priority)
```

```
    ACKNOWLEDGE vector
```

```
    CALL entry.serviceRoutine()
```

```
    RESTORE_INTERRUPT_MASK()
```

```
  END LOOP
```

```
END ALGORITHM
```

```
ISR STORAGE_SERVICE()
```

```
  DRAIN one completion; QUEUE deferred work
```

```
END ISR
```

```
ISR NETWORK_SERVICE()
```

```
  READ status; QUEUE received frame
```

```
END ISR
```

```
ISR TIMER_SERVICE()
```

```
  UPDATE tick; SCHEDULE deadline tasks
```

```
END ISR
```

Validation and Efficiency

Efficiency note: direct vector lookup removes repeated device-status polling. Priority masking is brief, and the deferred-work split keeps the processor responsive while still preserving interrupt order.

VALIDATION / EDGE CASES

Test or condition	Expected behaviour
Several interrupts arrive together	The controller supplies the highest-priority pending vector first.
Two devices share one priority	The controller retains both requests; the dispatcher handles the next one after return.
Unknown or invalid vector	Invoke the default error ISR, log the source, and return safely.
Long device processing	Queue deferred work so the ISR does not hold the CPU for an unbounded time.

RESULT: Vectored dispatch removes source polling and preserves priority order while keeping each interrupt service routine short.

QUESTION 3 — Dynamic Selection of I/O Addressing

Goal: choose the addressing mechanism from the current request profile rather than fixing one method for every device.

Problem Understanding

Inputs	Latency target, bandwidth demand, and address-space availability
Outputs	MEMORY_MAPPED_IO or IO_MAPPED_IO selection
Constraint	The selected mapping must pass a device-access verification check

Design Rationale

Memory-mapped I/O is selected for latency-critical or high-bandwidth register access because ordinary load and store instructions can access the device registers. This makes it suitable when frequent control and status reads must have low software overhead.

I/O-mapped I/O is selected for low-rate control traffic when conserving the processor memory address space is more important than using normal memory instructions. After either choice, the algorithm maps the registers and verifies a real device access before accepting the method.

Process Flow

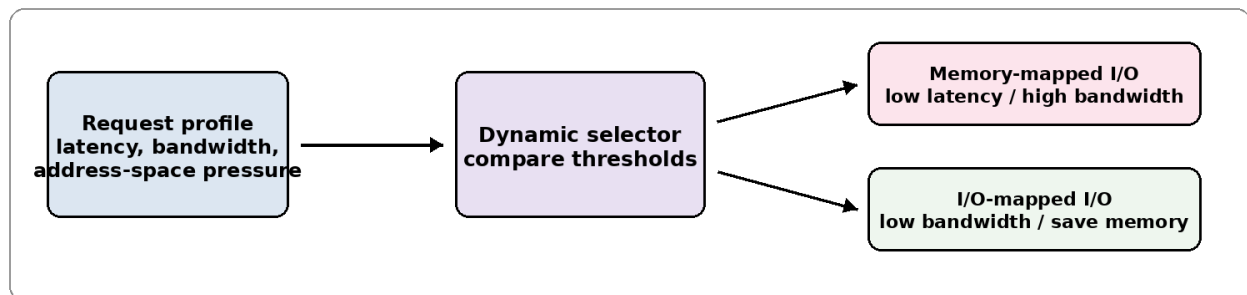


Figure 3. Runtime choice between memory-mapped and I/O-mapped access.

Execution Sequence

1. Read the request latency target, expected bandwidth, and current address-space availability.
2. Choose memory-mapped I/O when latency is strict or bandwidth is above the defined threshold.
3. Choose I/O-mapped I/O for lower-rate access when address space must be preserved.
4. Verify the mapped register access and undo the mapping if the verification fails.

Algorithm

PSEUDOCODE

```
ALGORITHM SELECT_IO_METHOD(request)
  REQUIRE request.latencyTarget > 0
  REQUIRE request.bandwidth > 0
  IF request.latencyTarget <= LOW_LATENCY_LIMIT THEN
    method <- MEMORY_MAPPED_IO
  ELSE IF request.bandwidth >= HIGH_BANDWIDTH_LIMIT THEN
    method <- MEMORY_MAPPED_IO
  ELSE IF MEMORY_ADDRESS_SPACE_IS_SCARCE() THEN
    method <- IO_MAPPED_IO
  ELSE
    method <- IO_MAPPED_IO
  END IF
  MAP_DEVICE_REGISTERS(request.device, method)
  IF VERIFY_ACCESS(request.device, method) = FALSE THEN
    UNMAP_DEVICE_REGISTERS(request.device)
    RETURN ERROR
  END IF
  RETURN method
END ALGORITHM
```

Validation and Efficiency

Efficiency note: the selector prefers memory-mapped access only where its lower software overhead matters. Lower-rate devices can use the separate I/O space, protecting valuable memory addresses for normal program data.

VALIDATION / EDGE CASES

Test or condition	Expected behaviour
Latency-critical request	Select memory-mapped I/O for efficient, low-overhead load/store access.
High-bandwidth device	Select memory-mapped I/O to reduce software access overhead.
Low-rate control with scarce address space	Select I/O-mapped I/O to preserve the memory address range.
Map or verification failure	Unmap the registers and return ERROR without using the device.

RESULT: The choice is dynamic: performance-sensitive devices use memory-mapped I/O, while economical low-rate control can use I/O-mapped access.

QUESTION 4 — Fair Multi-Interface Bus Arbitration

Goal: select requests across PCIe, USB, and Thunderbolt without missing urgent USB deadlines or starving bulk traffic.

Problem Understanding

Inputs	Interface queues, request deadlines, weights, credits, and wait time
Outputs	A bus grant with a bounded transfer quantum
Constraint	Urgent deadline traffic wins first; aging guarantees eventual service

Design Rationale

The arbiter treats USB periodic traffic as deadline-sensitive because missing a reserved service window can damage real-time behaviour. An imminent USB deadline therefore receives service before best-effort requests from the other interfaces.

When no deadline is urgent, weighted round-robin distributes bus time across PCIe, USB, and Thunderbolt. Aging increases the priority of a request that has waited too long, so a busy high-bandwidth source cannot starve another interface indefinitely.

Process Flow

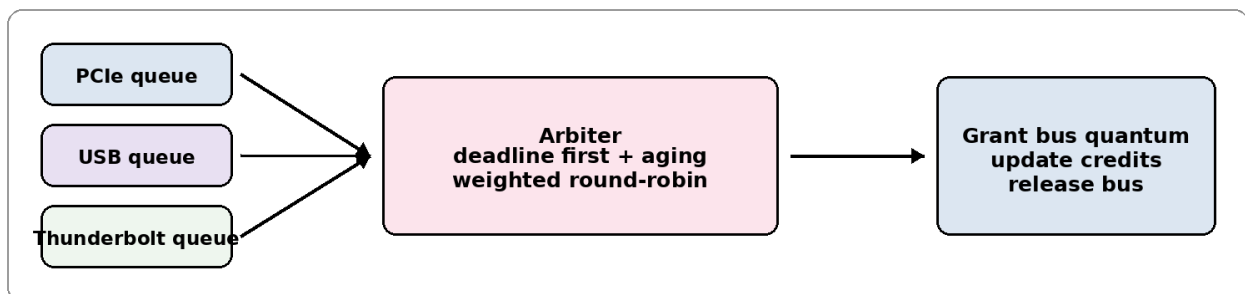


Figure 4. Deadline, aging, and weighted round-robin arbitration.

Execution Sequence

1. Increase wait time for every queued request and first check for an urgent USB deadline.
2. If no deadline is urgent, promote aged requests before running weighted round-robin.
3. Grant only a bounded quantum, then update interface credits and request state.
4. Release the grant so the arbiter can reconsider priorities and deadlines fairly.

Algorithm

PSEUDOCODE

```
ALGORITHM ARBITRATE_BUS(requestQueues)
  LOOP WHILE any queue is nonempty
    INCREMENT_WAIT_TIME(requestQueues)
    urgent <- EARLIEST_DEADLINE_USB_REQUEST()
    IF urgent exists THEN
      selected <- urgent
    ELSE
      PROMOTE requests waiting beyond AGE_LIMIT
      selected <- WEIGHTED_ROUND_ROBIN(
        PCIe, Thunderbolt, USB)
    END IF
    IF selected = NONE THEN
      WAIT_FOR_NEW_REQUEST()
      CONTINUE
    END IF
    GRANT selected.interface
    TRANSFER selected.quantum
    UPDATE_CREDITS(selected.interface)
    UPDATE_DEADLINE(selected)
    RELEASE_BUS_GRANT()
  END LOOP
END ALGORITHM
```

Validation and Efficiency

Efficiency note: bounded transfer quanta prevent one high-throughput interface from retaining the bus indefinitely. Credits and aging make the arbiter work-conserving without sacrificing fairness or deadline protection.

VALIDATION / EDGE CASES

Test or condition	Expected behaviour
Urgent periodic USB request	Serve the earliest deadline before best-effort traffic.
No urgent deadline	Use weighted round-robin across PCIe, Thunderbolt, and USB.
Long-waiting PCIe or Thunderbolt request	Promote after AGE_LIMIT, then grant a weighted quantum.
No selected request after queue scan	Wait for a new request instead of issuing an invalid grant.

RESULT: Deadline protection, bounded quanta, weighted rotation, and aging together provide both responsiveness and fairness across all interfaces.

QUESTION 5 — Hybrid Polling and DMA-with-Interrupts

Goal: poll inexpensive low-speed devices on a fixed interval and use DMA plus interrupts for continuous high-speed data.

Problem Understanding

Inputs	Low-speed and high-speed lists, poll interval, DMA rings, and buffers
Outputs	Processed low-rate items and queued high-rate completed buffers
Constraint	The ISR only acknowledges/requeues; a worker performs data processing

Design Rationale

Polling is efficient for low-speed devices because the controller performs a quick status check at a predictable interval and avoids the overhead of frequent interrupts. A device that is not ready simply waits for the next poll without blocking the whole system.

High-speed devices use DMA rings because frequent payload transfers would otherwise force the CPU to copy data repeatedly. Completion interrupts notify the controller only when a buffer is ready, and the ISR immediately queues the buffer to a worker before posting a replacement buffer.

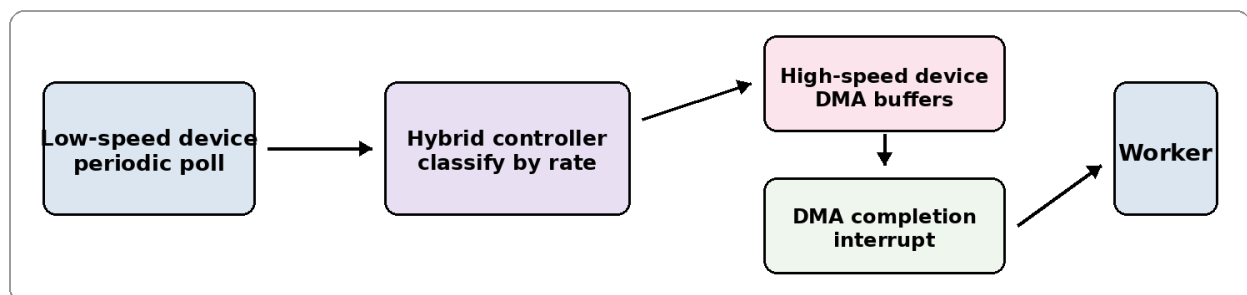
Process Flow

Figure 5. Polling for low-rate traffic and interrupt-driven DMA for high-rate traffic.

Execution Sequence

1. Poll each low-speed device once per interval and process only ready items.
2. Keep a DMA buffer queued for every active high-speed device.
3. Handle a DMA completion with a short ISR that acknowledges, queues, and replaces the buffer.
4. Move expensive parsing or application processing to the worker after the ISR returns.

Algorithm

PSEUDOCODE

```
ALGORITHM HYBRID_IO_SERVICE(lowSpeed, highSpeed)
  CONFIGURE DMA rings for highSpeed
  ENABLE completion interrupts
  LOOP FOREVER
    FOR each device IN lowSpeed DO
      IF POLL_STATUS(device) = READY THEN
        item <- READ(device)
        PROCESS item
      END IF
    END FOR
    FOR each device IN highSpeed DO
      IF DMA_IDLE(device) THEN
        QUEUE_DMA_BUFFER(device)
      END IF
    END FOR
    WAIT_FOR_EVENT_OR_TIMEOUT(POLL_INTERVAL)
  END LOOP
END ALGORITHM
```

```
INTERRUPT_HANDLER HIGH_SPEED_DMA_ISR(device)
  ACKNOWLEDGE interrupt
  completed <- GET_COMPLETED_BUFFER(device)
  IF completed = INVALID THEN
    RECORD_ERROR(device)
    RETURN_FROM_INTERRUPT
  END IF
  QUEUE completed FOR worker
  QUEUE_DMA_BUFFER(device)
  RETURN_FROM_INTERRUPT
END HANDLER
```

Validation and Efficiency

Efficiency note: polling is restricted to inexpensive, low-rate checks. DMA handles the continuous payload path, and the short ISR merely changes buffer ownership before a worker processes the data.

VALIDATION / EDGE CASES

Test or condition	Expected behaviour
Low-speed device is not ready	Continue without blocking and poll the device at the next interval.
DMA completion interrupt	Queue the finished buffer, start a replacement buffer, and return quickly.
Invalid completed DMA buffer	Record the device error and return without passing invalid data to the worker.
High-speed device becomes idle	Queue the next DMA buffer so the continuous transfer path can resume.

RESULT: The hybrid design keeps low-speed overhead predictable while DMA and short interrupts protect the CPU from high-speed copy work.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient